

AI IN Engineering HACKATHON | TRACK: SYSTEM MODELLING

SpecAlive

Hackathon to develop Agentic AI Product for System Modelling

FROM SPECS TO LIVE ENGINEERING MODELS

Contents

1. Summary.....	3
2. Why this problem	3
3. Objective.....	3
4. Intended users.....	3
5. Scope	4
5.1 Minimum scope.....	4
5.2 Suggested extensions.....	4
5.3 Out of scope	4
6. Inputs	4
6.1 Modalities.....	5
6.2 Properties the input will have	5
6.3 Required input handling behaviour.....	5
7. Expected output	5
8. Acceptance criteria.....	6
8.1 Speed	6
8.2 Accuracy	6
8.3 Scope	7
9. Benchmark test cases.....	7
9.1 What the cases are and are not.....	7
10. Architecture considerations	8
11. Submission.....	8
12. Evaluation alignment.....	8
13. References	9

If the contents list above appears blank, select all and press F9 in Word to refresh it.

1. Summary

Systems engineers spend a large share of their modelling effort on translation rather than on engineering. A plant description arrives as a paragraph of prose, a scanned P&ID, a whiteboard photograph, a requirements table in a spreadsheet, and a vendor datasheet in PDF. Turning that pile into a structured system model, and then into something that simulates, is manual, slow, and dependent on the modeller's own conventions.

This challenge asks participants to build an AI product that closes that gap. The system accepts mixed, unstructured engineering inputs and produces a structured system model, expressed in SysML, and a corresponding executable Modelica model that compiles and simulates.

Four reference engineering problems are supplied as unit test cases. They are a calibration set, not the product. A submission that only solves the four supplied problems has built a demo. A submission that solves them because its architecture generalises has built a product. The evaluation is weighted accordingly.

2. Why this problem

Every user will arrive with different inputs, different notation, different levels of completeness, and different house conventions. That variability is precisely why rule-based parsers and fixed templates have historically failed at this task, and precisely why language models are a credible approach now.

The interesting engineering question is not whether a language model can emit Modelica syntax. It is how to architect a system that stays correct, traceable, and honest about what it does not know, when the input is ambiguous and the output must be physically valid. That question is the point of the challenge.

3. Objective

Build a product that takes engineering inputs in natural form and returns a system model that an engineer can review, trust, and run in Modelica tools.

Success means an engineer can hand the tool a messy description of a physical system and get back, without hand-writing the model, a structural model they recognise as correct and a simulation they can execute.

4. Intended users

User	Need
Systems engineer	Convert a specification or sketch into a first-pass SysML model without starting from a blank canvas
Simulation engineer	Get a Modelica skeleton that compiles, so effort goes into physics and tuning rather than boilerplate
Design reviewer	See the assumptions the model made explicit, and trace each element back to the input that justified it
Domain newcomer	Produce a structurally legal model without deep tool expertise

Teams may narrow to a single user and build depth rather than breadth. State which user you chose.

5. Scope

5.1 Minimum scope

Required for a submission to be considered valid.

1. Accept at least one input modality end to end, with text as the baseline.
2. Produce a structured intermediate representation of the system: parts, ports, connections, attributes.
3. Emit a SysML representation of that model.
4. Emit a Modelica model that compiles without error in OpenModelica or an equivalent tool.
5. Demonstrate the full path on at least one of the four supplied test cases.
6. List assumptions made and information found missing.

Anything beyond this is yours to define. Teams are explicitly free to extend into software architecture, system design, product design, user experience, deployment, or evaluation tooling. Scope expansion is rewarded when it is coherent, not when it is merely large.

5.2 Suggested extensions

- Additional input modalities: images, scanned drawings, PDFs, spreadsheets, CAD geometry, 3D layout, or mechanical drawing interpretation.
- Clarifying dialogue, where the system asks the user targeted questions instead of guessing.
- State machine and interlock generation, not just structure.
- Simulation execution with plotted results.
- Model repair, where a compile error is fed back and corrected autonomously.
- Round-trip editing, where a user's correction updates the model and the traceability record.
- Confidence scoring per generated element.
- A library or reuse mechanism, so a component modelled once is recognised again.

5.3 Out of scope

- Proprietary or confidential plant data of any kind. Use only the supplied cases or material you are free to share.
- Hardware-in-the-loop or real-time control deployment.
- Chamber-level or plant-level industrial benchmarks requiring curated internal engineering data. These sit beyond this hackathon by design.

6. Inputs

The product must assume inputs are unstructured, inconsistent, and incomplete. There is no fixed input schema, and there will not be one.

6.1 Modalities

Modality	Examples
Text	Prose description of a plant, an operating procedure, a numbered requirements list, an email thread, a pasted specification
Image	Photograph of a whiteboard, hand-drawn sketch, scanned P&ID or block diagram, screenshot of a schematic
Document	PDF specification or datasheet, Word requirements document, spreadsheet of parameters, tabular component list
Mixed bundle	Any combination of the above describing one system, possibly with overlaps and contradictions, Even 3D cad model, P&ID diagrams, drawing files etc.

6.2 Properties the input will have

- Incomplete. Parameters, units, and initial conditions will be missing.
- Ambiguous. The same component may be named differently in two places.
- Contradictory. A diagram and a paragraph may disagree.
- Implicit. Engineering conventions will be assumed rather than stated, for example that a tank has an outlet even when the text never says so.
- Custom. Every user will bring their own notation and vocabulary.

How the system behaves under each of these is a first-class part of the evaluation, not an edge case.

6.3 Required input handling behaviour

- Missing information must be either inferred with a stated assumption, or surfaced as a question. It must not be silently invented.
- Contradictions must be flagged rather than resolved arbitrarily.
- The system must not fabricate components, ports, or physics that are not derivable from the input or from a declared standard assumption.

7. Expected output

Output is layered. A submission need not deliver every layer, but the layers it does deliver must be consistent with one another.

Layer	Name	Content
1	Structured system representation	Parts, ports, connections, attributes and units, and where applicable states and transitions. Format is the team's choice: JSON, YAML, or a defined schema. This layer exists so the SysML and Modelica outputs come from a single source of truth.

Layer	Name	Content
2	SysML model	SysML v2 textual is must. Covers structural decomposition, ports and interfaces, connections, and behaviour where required.
3	Evidence and traceability	Assumptions made, missing or ambiguous information, traceability from input fragment to model element, and simulation output where available.
4	Human-readable summary	The system as the model understands it, suitable for an engineer to skim and approve or reject in under a minute.
5	Modelica model	A .mo model of the same system. It must compile. Where the problem is well posed enough to run, it should simulate and produce plausible trajectories. Use of the Modelica Standard Library is encouraged for libraries.

Note: Generating SysML and Modelica independently is the most common way to get silent divergence between them.

8. Acceptance criteria

Exact numeric targets are deliberately loose. These are thresholds for a credible submission, not a scoring rubric, and judges will weigh reasoning quality alongside them.

8.1 Speed

Stage	Target
First structural draft from a text input	Under 2 minutes
Full pipeline to compiling Modelica, single test case	Under 10 minutes*
End-to-end live demonstration during judging	Within the allotted demo slot, on a normal laptop or standard API access

Latency that comes from genuine multi-step reasoning or self-correction is acceptable and will not be penalised on its own. Silence with no progress indication will be.

* Provided cases should be completed within the target time.

8.2 Accuracy

Dimension	Expectation
Structural correctness	Parts, ports, and connections materially match the reference for the attempted case. Aim for 80 percent or better element coverage on L1 and L2
Compilability	The generated Modelica compiles without error. This is a hard gate, not a target
Simulatability	For L1 and L2, the model runs and produces physically plausible behaviour

Dimension	Expectation
Behavioural correctness	Where a state machine is required, required states are reachable and forbidden combinations are prevented
Repeatability	The same input run twice yields a structurally equivalent model. Naming variation is acceptable, topology variation is not
Honesty	No fabricated components. Every element traces to an input fragment or a declared assumption

A model that is partially complete and honest about its gaps scores above one that is superficially complete and quietly wrong. This is deliberate.

8.3 Scope

- One test case solved end to end beats four solved partially.
- Generalisation beats case-specific handling. If your pipeline contains logic that only fires for the two-tank problem, say so, because judges will look.
- Depth in one input modality beats shallow coverage of all three.

9. Benchmark test cases

The four problems below are supplied as unit test cases. They form a deliberate difficulty ladder, where each level adds one new class of reasoning rather than a new subject area. Detailed problem descriptions and reference material are provided separately.

Level	Case	Reasoning skill under test
L1	Two-Tank Controller	Sequencing, interlocks, state machines, safe start and stop. Structural basics, no physics coupling
L2	Room CO2 Ventilation Controller	Closed-loop feedback: disturbance, sensor, control law, actuator acting back on the process
L3	Magnetic Circuit	Multiphysics energy transfer across domains, with an explicit loss path
L4	Water and NaCl Evaporation Plant	Integration: material supply, heating, transformation, separation into two streams, coordinated under one controller

L4 is not a new skill. It is the composition of the previous three under competing constraints, which is why it is the hardest.

9.1 What the cases are and are not

These problems are proxies, chosen because their reasoning patterns are structurally faithful to far more complex industrial equipment while remaining cheap to author, score, and run inside a hackathon. Valve sequencing in a two-tank rig is the same pattern as sequencing in a process gas delivery chain. A CO2 loop

is the same pattern as chamber pressure control. **Do not optimise for tanks and brine. Optimise for the pattern.**

10. Architecture considerations

Not prescriptive. Offered because these are the decisions judges will ask about.

- Where does the language model sit: end-to-end generation, or extraction into a schema followed by deterministic code generation? The second is usually more reliable and easier to defend.
- How do you validate before showing the user: compile-check, schema-validate, unit-check, or simulate?
- What happens on failure: retry, repair loop, degrade gracefully, or ask the user?
- How is agentic behaviour used? Planning, tool use, self-critique and multi-pass refinement are all in scope. Using an agent where a single call would do is not automatically better.
- How do you keep the three output layers consistent with one another?
- How is domain knowledge injected: prompting, retrieval over a component library, fine-tuning, or a typed schema that constrains what can be generated?

11. Submission

Item	Requirement
Working artefact	Runnable code or a hosted demo, with setup instructions. SaaS is preferred
Generated outputs	SysML and Modelica files for each attempted test case, plus simulation output where available
Architecture note	Short document or diagram showing the pipeline and the key design decisions
Assumptions log	What the system assumed, and what it could not resolve
Demonstration	Live or recorded walkthrough on at least one test case

Submission mechanism, format, and deadline are covered in the hackathon announcement.

12. Evaluation alignment

This problem is designed to exercise all three learning outcomes the hackathon is targeting.

Outcome	How this problem tests it
Domain understanding	Whether the generated model is physically and structurally sensible, not just syntactically valid
AI architecture	How the pipeline is composed, validated, and made to fail safely, and whether the agentic design is justified

Outcome	How this problem tests it
Design thinking	Who the product is for, how ambiguity is surfaced to the user, and whether an engineer would actually trust and use it

13. References

- System Modelling Hackathon: Benchmark Rationale. Stakeholder brief describing the selection of the four benchmark problems and the reasoning ladder they form.
- Individual problem descriptions for L1 to L4, issued separately.
- SysML v2 specification and pilot implementation documentation.
- Modelica Language Specification and Modelica Standard Library documentation.
- OpenModelica user documentation.

Draft for review. Please raise corrections against sections 5, 7 and 8 first, since those define what participants are actually held to.